

Loading and Saving Spreadsheet Data

Cpt S 321 Homework Assignment

Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

1. Create a branch called "Branch_HW9" and work in this branch for this assignment.
2. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
3. Work on the Assignment Instructions below.
4. Create a pdf document with your AI statement for the HW (call it "AI_HW9"), stating whether or not you used GenAI and how and upload it in the "AI_Statements" folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.
5. When you are done, merge the branch back to the master.
6. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, "HW9".
7. Record a video to demo your HW (call it "Demo_HW9-FirstName_LastName").
8. On Canvas -> Assignments -> Upload your pre-recorded video by the HW deadline.
Don't forget to sign up and demo with the TAs.

Assignment Instructions:

Read each step's instructions *carefully* before you write any code.

In this assignment, you will add loading and saving capabilities to your spreadsheet application. If you created a Workbook class for the previous assignment (or would like to create one for this assignment) then add Load and Save methods to it. Otherwise you can add these methods to your Spreadsheet class.

Design an XML format for your spreadsheet data. At a high level, it will probably have a structure somewhat like:

```
<spreadsheet>
  <cell name="B1">
    <bgcolor>FF8000FF</bgcolor>
    <text>=A1+6</text>
  </cell>
</spreadsheet>
```

You'll obviously have more than one cell in most cases. Make sure you do the following:

- Provide saving and loading functions that take a stream as the parameter.
- Add menu options in the UI for saving and loading.
- Make sure the saving and loading code is in the logic engine.
- Use existing XML classes from the .NET framework.
- When saving, only write data from cells that have one or more non-default properties. This means that if a cell hasn't been changed in any way then you don't need to write data for it to the file.
- Clear all spreadsheet data before loading file data. The load-from-file action is NOT a merge with existing content.
- Clear the undo/redo stacks after loading a file.
- Make sure formulas are properly evaluated after loading.
- You may assume only valid XML files will be loaded, but make sure loading is resilient to XML that has different ordering from what your saving code produces as well as extra tags. As a simple example, if you're always writing the <bgcolor> tag first for each cell followed by the <text> tag, then your loader must still support files that have these two in the opposite order. Also, if you didn't write more than these two tags within the <cell> content, your loader should just ignore extra tags when loading. See the example below.
- Use XML reading/writing capabilities from .NET. Do not write your own XML parsers that do things manually down at the string level. We discussed several options in class, such as [XDocument](#), [XmlDocument](#), [XmlReader](#), and [XmlWriter](#).

If you saved:

```
<spreadsheet>
  <cell name="B1">
    <bgcolor>FF8000</bgcolor>
    <text>=A1+6</text>
  </cell>
</spreadsheet>
```

then you must be able to load:

```
<spreadsheet>
  <cell unusedattr="abc" name="B1"> <text>=A1+6</text>
    <some_tag_you_didnt_write>blah</some_tag_you_didnt_write>
    <bgcolor>FF8000</bgcolor> <another_unused_tag>data</another_unused_tag>
  </cell>
</spreadsheet>
```

Homework Grading Rubric and Points Breakdown

CR1: Functionality (4 points)	● Fulfill all the requirements above with no inaccuracies in the output and no crashes.
CR2 Quality of Version Control (1 point)	● Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). ● Each commit should have cohesive functionality. ● Commit messages should concisely describe what is being committed. ● TDD should be used (i.e, write and commit tests first and then implement and commit functionality). ● Include “TDD” in all commit messages with tests that are written before the functionality is implemented. ● Use of a .gitignore. ● Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	● No underscores in names of classes, attributes, and properties. ● No numbers in names of classes or tests. ● Identifiers should be descriptive. ● Project names should make sense. ● Class names and method names use PascalCasing. ● Method arguments and local variables use camelCasing. ● No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	● Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. ● All comment blocks use the format that is generated when typing “///” on the line above each entity. ● There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	● Each file should only contain one public class. ● Correct use of access modifiers. ● Classes are cohesive. ● Namespaces make sense. ● Code is easy to follow. ● StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. ● Use of appropriate design patterns and software principles seen in class.

CR6: Existence and Quality of Tests (1 point)	• Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. • Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). • <i>Note: In assignments with a GUI, we do not require testing of the GUI itself.</i>
CR7: Demo video – 5 min max (1 point)	• Check the “Pre-recorded demos instructions” in the additional resources for details on the items below. • Version control history overview (< 30 sec.) • AI statement (< 30 sec.) • Design of the solution (2 min max) • Features showcase (2 min max)